Agent Harness 工作坊

从零构建生产级 AI Agent

动手实操 | 6 个递进式实验 | 约 3 小时

核心洞察： **Agent = 模型 + Harness**

Harness 六层架构

将原始大模型转化为可靠的生产级 Agent 的六个核心层

01 提示与引导层

系统提示词、钩子、行为规则

02 工具与执行层

统一工具接口、沙箱执行

03 安全与验证层

权限中间件、人机协同、自动测试

04 上下文与记忆层

微压缩、会话持久化

05 规划与协调层

任务分解、多 Agent 调度

06 状态与持久层

工作区存储、产物管理

模型能力正在趋同 —— Harness 才是竞争力的分水岭。

实验 1

提示与引导层

30 分钟

痛点

裸模型行为不可预测 —— 不遵守项目规范、不理解自身角色、无法拒绝不安全请求。

你将构建

- 系统提示词组装（角色 + 规则注入）
- CLAUDE.md 项目记忆自动发现
- 钩子生命周期（PreToolUse / PostToolUse）
- 后续所有实验的基础 Agent Loop

关键概念

提示词工程 | 项目记忆 | 钩子生命周期

成果： Agent 遵循一致的行为规则，在执行前理解项目上下文。

实验 2 工具与执行层

35 分钟

痛点

Agent 有引导但无法行动 —— 工具调用只生成文本，无法真正执行文件读写或命令。

你将构建

- 统一工具接口（抽象基类）
- 三个核心工具： ReadFile 、 WriteFile 、 Bash
- 执行流水线：校验 → 执行 → 返回结果
- Anthropic API 的 JSON Schema 工具定义

关键概念

工厂模式 | 工具 Schema | 输入校验

成果： Agent 获得真正的 '手脚' —— 读写文件、执行 Shell 命令。

实验 3

安全与权限层

30 分钟

痛点

Agent 能执行操作但零安全检查 —— 如果被要求执行 'rm -rf /' ，它会毫不犹豫地执行。

你将构建

- 三级权限控制（允许 / 拒绝 / 询问）
- Bash 命令分类器 + 模式检测
- 人机协同（HITL）确认系统
- 危险模式正则与安全规则引擎

关键概念

中间件模式 | 命令分类 | HITL 流程

成果：安全执行 —— 'ls' 自动放行，'sudo rm' 需要人工确认。

实验 4

上下文与记忆层

25 分钟

痛点

长对话导致 Token 爆炸；重启 Agent 后所有状态和上下文全部丢失。

你将构建

- 微压缩：压缩旧输出、保留近期上下文
- Token 估算与上下文窗口管理
- 项目记忆系统（CLAUDE.md 自动注入）
- 跨会话知识持久化

关键概念

Token 管理 | 上下文压缩 | 记忆分层

成果： Agent 处理长会话不超限，跨重启保留项目知识。

实验 5

规划与协调层

25 分钟

痛点

复杂多步任务迫使单个 Agent 在同一上下文中处理一切 —— 无法分解、委派或并行。

你将构建

- AgentTool ：生成独立上下文的子 Agent
- 任务管理系统（创建 / 列表 / 更新）
- SendMessageTool ： Agent 间通信
- 递归生成防护机制

关键概念

领队模式 | 上下文隔离 | 任务追踪

成果：多 Agent 协调 —— 主 Agent 分解任务并调度子 Agent 。

实验 6

状态与持久层

20 分钟

痛点

退出程序后一切丢失 —— 对话历史、任务进度、所有累积的上下文全部消失。

你将构建

- SessionStore ：消息历史持久化为 JSON
- 恢复机制：无缝加载历史会话
- 回退能力：回滚到更早的对话状态
- UUID 会话 ID + 文件检查点

关键概念

会话快照 | 消息序列化 | 恢复流程

成果：完整 Mini Harness —— Agent 可恢复中断的工作并回滚错误。

工作坊总结

从裸模型到生产级 Agent ，6 步递进式构建

1. 提示与钩子 —— 行为控制
2. 工具执行 —— 真实操作
3. 安全层 —— 权限与人机协同
4. 上下文与记忆 —— 长会话支持
5. 多 Agent —— 分解与委派
6. 持久化 —— 恢复与回退

~3 小时 总时长	**6 个** 动手实验
Python 实现语言	**100%** 实操导向

模型是大脑。 **Harness 是其他一切。**